



GNC Simulations with Basilisk: User Guide

Rodrigo Dias
2025/2026

Contents

1	Objective	2
2	Introduction: What is Basilisk?	2
3	Architecture	2
3.1	Code	2
3.2	Execution Hierarchy	2
4	Code	3
4.1	Use cases	3
4.1.1	Perturbations	3
4.1.2	Thruster and reaction wheel configurations	3
4.2	Default controllers	3
4.3	Configuration File	3
4.3.1	use_cases	3
4.3.2	fsw_settings	4
4.4	sat_params	4
4.5	actuator_params	4
4.5.1	thruster_orient and thruster_main	4
4.6	reaction_wheel	5
5	Custom External Controller Integration	5
5.1	Controller Structure and Requirements	5
5.1.1	Inputs	5
5.1.2	Outputs	6
6	How to run	6

1 Objective

The objective of this document is to succinctly explain the use of the code created in the Basilisk software for GNC system controllers within the Basilisk software.

2 Introduction: What is Basilisk?

Basilisk, or BSK for short, is a software capable of performing faster-than-real-time spacecraft simulations, including repeatable Monte Carlo simulation options, as well as providing real-time options for *hardware-in-the-loop* simulations. The software is being developed jointly by the University of Colorado's AVS Lab and the Laboratory for Atmospheric and Space Physics (LASP).

3 Architecture

3.1 Code

Basilisk uses *python* and C/C++ to achieve high efficiency. Communication between these languages is handled by SWIG, which translates between them. In this way, the heavier code with a higher volume of calculations and operations is written in C/C++, while program control is handled in *python*. In other words, simulations are built in *python*, where the scenario and the system's initial conditions are defined. Then, the C/C++ functions are called, the results are returned, and based on them, graphs are plotted in *python* and data analysis is performed.

3.2 Execution Hierarchy

The code is organized into three levels: processes, tasks, and modules:

Processes: Sets of Tasks. For example, in a simulation with multiple satellites, it would be possible to create a Process for each satellite in order to keep the associated tasks and embedded modules organized.

Tasks: A set of modules that are executed at the same update rate.

Modules: Framework functions. For example, the spacecraft module allows defining the mass, center of mass, and moments of inertia of the satellite; with these data and the forces applied to the system, it calculates the new positions and velocities using the equations of motion.

It is possible to define execution priority levels among processes, tasks, and modules.

4 Code

4.1 Use cases

To facilitate the setup of different simulation scenarios, the code includes predefined use cases. These allow the user to easily configure the desired simulation by selecting from various perturbation levels, thruster and reaction wheel configurations, and controller cases.

4.1.1 Perturbations

The environment supports three distinct levels of perturbations:

- Level 0: no perturbations, only ideal spherical gravity of celestial bodies¹.
- Level 1: Earth J2 perturbation and spherical gravity of other celestial bodies
- Level 2: all perturbations at the same time: Earth J2, solar radiation pressure, and drag force

4.1.2 Thruster and reaction wheel configurations

The code implements 3 different possible configurations for thrusters and reaction wheels:

- Configuration 0: 2 thrusters with opposite directions per axis.
- Configuration 1: reaction wheels for attitude control and a higher capacity thruster for translation.
- Configuration 2: same as configuration 1 but reaction wheels are replaced by chemical thrusters

4.2 Default controllers

There are 3 different control strategies depending on the simulation profile:

- Attitude controller: Exclusively satellite orientation.
- Rendezvous controller: Manages the proximity operations and docking sequence of a chaser satellite approaching an uncooperative or passive target satellite.
- Relative Orbit and Formation Flying controller: Maintains a constant relative orbit for a cluster of 4 satellites², ensuring precise formation flight.

4.3 Configuration File

The use cases and simulation configurations are defined in the configuration file, **Config.json**. This file is composed of multiple parameter groups:

4.3.1 use_cases

This parameter group is used to choose the use cases and scenario deployment, it has the following variables:

- "perturbCase": 0 to 2.
- "thrusterCase": 0 to 2.

¹The celestial bodies defined are: Earth, Sun, Moon, Jupiter barycenter and Mars barycenter

²The architecture is parameterized to easily scale the number of satellites in the formation.

- "controlCase": 0 to 2.
- "number_of_satellites": Sets the fleet size for **controlCase** 2.

4.3.2 fsw_settings

This group defines the Flight Software algorithm source aswell as the simulation time:

- "sim_time": the simulation type in minutes.
- "controller_type": Set to "CUSTOM" to enable the external Python bridge. Alternatively, set to "MRP" to use the standard Basilisk control strategies associated with each **controlCase**, which are largely based on Modified Rodrigues Parameter feedback control.
- "custom_controller_file": Name of the external .py file (without extension). This parameter is only mandatory when "*controller_type*" is set to "CUSTOM".

4.4 sat_params

The satellite parameters are defined in this group:

- "ModelTag": the name chosen for the satellite module.
- "mass": mass of the satellite (kg).
- "center_of_mass": center of mass of the satellite in the satellite reference frame³.
- "inertia": inertia matrix in the satellite reference frame ($\text{kg} \cdot \text{m}^2$).
- "initial_position": initial position in the inertial reference frame (m).
- "initial_velocity": initial velocity in **N** (m/s).
- "MRPs": initial modified Rodrigues parameters in **B**.
- "angular_velocity": initial angular velocity in **B**.
- "drag_projectedArea": satellite projected cross-sectional area subject to atmospheric drag (m^2).
- "drag_dragCoeff": atmospheric drag coefficient.
- "srp_area": satellite surface area subject to solar radiation pressure (m^2).
- "srp_coefficientReflection": solar radiation pressure coefficient.

4.5 actuator_params

In the existing thruster and reaction wheel configurations there are 3 actuators: attitude thrusters, a translational thruster with a bigger thrust in comparison to the attitude ones and reaction wheels for attitude. The "d" parameter is the distance from the satellite center of mass to the attitude control thrusters, acting as the moment arm to generate rotational torque (m).

4.5.1 thruster_orient and thruster_main

- "MaxThrust": maximum force output that the thruster can produce when fully active (N).
- "MinOnTime": minimum duration for which the thruster valve must remain open during a single pulse command to prevent hardware wear and guarantee stable propellant ignition (s).

³In Basilisk the satellite reference frame is designated as **B** and the inertial reference frame as **N**, it's origin is in the center of mass of the central celestial body configured as Earth in this simulation

- "steadyIsp": the steady-state specific impulse of the thruster, representing its fuel efficiency or how much thrust is produced per unit rate of propellant mass consumption (s).
- "areaNozzle": the cross-sectional exit area of the thruster's exhaust nozzle (m^2).

4.6 reaction_wheel

- "mass": mass of the reaction wheel assembly (kg).
- "Omega_max": the maximum allowable spin rate of the reaction wheel flywheel (rpm).
- "Omega": the initial spin rate of the reaction wheel flywheel (rpm).
- "u_max": the maximum motor torque limit that the reaction wheel can output to control the satellite ($\text{N} \cdot \text{m}$).
- "u_min": the minimum torque threshold or resolution limit for the reaction wheel motor ($\text{N} \cdot \text{m}$).
- "maxMomentum": the maximum angular momentum storage capacity of the reaction wheel before saturation ($\text{N} \cdot \text{m} \cdot \text{s}$).
- "fCoulomb": the static Coulomb friction torque acting on the reaction wheel bearing axis ($\text{N} \cdot \text{m}$).

5 Custom External Controller Integration

The simulation architecture supports the integration of user-defined, pure Python controllers without requiring modifications to the main simulation scripts. This is achieved through a dynamic communication bridge that links the Basilisk navigation sensors to the custom control algorithm and routes its outputs back to the physical actuators.

5.1 Controller Structure and Requirements

The external Python file must strictly adhere to a specific format to interface correctly with the simulation loop. It must contain a main function named **evaluate_controller**, which is automatically called by the C++ background processes at every flight software update step.

5.1.1 Inputs

The function must accept exactly five arguments in the following order, representing the dynamic and configuration state of the spacecraft:

- **sigma**: A list of 3 values representing the current orientation in Modified Rodrigues Parameters (MRPs).
- **omega**: A list of 3 values representing the angular velocity vector in the body reference frame (rad/s).
- **r_BN**: A list of 3 values representing the inertial position vector (m).
- **v_BN**: A list of 3 values representing the inertial velocity vector (m/s).
- **thruster_case**: An integer representing the current active **thrusterCase** (0, 1, or 2), allowing the algorithm to adjust actuator mapping dynamically.

5.1.2 Outputs

The user's control logic computes the required maneuvers based on the inputs and must return exactly two lists to be processed by the actuators. These must be returned in the following order:

- **ideal_torque**: A list of 3 values representing the required continuous control torque vector $[x, y, z]$ ($N \cdot m$). This output is automatically routed to the reaction wheels if **thrusterCase** 1 is active.
- **calculated_ontimes**: A list of valve firing durations (s). The size of this list must adapt dynamically based on the input *thruster_case*:
 - *thruster_case* 0: Size 6 (6 attitude thrusters).
 - *thruster_case* 2: Size 7 (index 0 for main translational thruster, indexes 1-6 for attitude thrusters).

All values must respect the physical simulation time step limits.

6 How to run

In order to run the code you must have the Basilisk software downloaded in your pc. It is downloaded with the following command in ur terminal:

```
pip install bsk
```

To run the simulation, you need to put all the files in the same folder and simply execute the Main.py file. This will generate a JSON file containing the simulation data, specifically the spacecraft's position, velocity, angular velocity (along with its errors), and attitude orientation (along with its errors). It will also generate a .bin file that can be used to visualize the simulation in 3D with the Vizard application.